

Does Optimization Parallel Inference Change LLM Inference Outputs?

Xudong Zhu

May 9, 2026

Code: <https://github.com/xzAscC/CSE5429>

TL;DR

We test whether vLLM system-level inference optimizations change outputs for two Qwen models: Qwen2-1.5B-Instruct and Qwen3-1.7B. For both models, chunked prefill, prefix caching, low GPU memory, CPU offload, and repeated baseline runs match the baseline exactly on all 20 prompts. CUDA graphs and batch processing are the only configurations that change outputs. The qualitative pattern is stable across both models, but the magnitude differs: CUDA graphs match 25% of Qwen2 baseline outputs and 45% of Qwen3 baseline outputs; batch processing matches 50% for Qwen2 and 30% for Qwen3.

1 Question and Motivation

LLM serving systems use optimizations such as chunked prefill, CUDA graphs, prefix caching, batching, and CPU offloading to improve throughput and latency. These optimizations are usually treated as implementation details: they should make inference faster, not change generated text. This report tests that assumption directly across two Qwen-family models.

The core question is: *given the same model, prompt, seed, and greedy decoding setup, do different inference configurations produce the same output?* If outputs change when the serving path changes, then inference configuration is part of the experimental condition, not merely an engineering detail.

2 Experimental Setup

We run the same experiment on Qwen/Qwen2-1.5B-Instruct and Qwen/Qwen3-1.7B using vLLM v0.20.1 on a single NVIDIA RTX 4090. All runs use greedy decoding with temperature = 0, seed = 42, maximum context length 2048, and maximum generation length 128. The prompt set contains 20 prompts spanning short factual, medium creative, long reasoning, code-generation, edge-case, and repetition-sensitive inputs. Each inference configuration runs in an isolated subprocess to avoid GPU-memory state carrying over between configurations.

Table 1 lists the remaining nine configurations.

Outputs are compared with exact match, BLEU, edit distance, and first divergent token position. Exact match is the primary metric because it directly answers whether two generations are character-for-character identical.

Table 1: Inference configurations used for both models.

ID	Name	Purpose
C0	Baseline	Reference configuration with eager execution
C1	Chunked Prefill (512)	Tests small prefill chunking
C2	Chunked Prefill (2048)	Tests larger prefill chunking
C3	CUDA Graphs	Tests CUDA graph execution rather than eager mode
C4	Prefix Caching	Tests KV cache prefix reuse
C5	Batch Processing	Tests batched prompt scheduling
C6	Low GPU Memory	Tests lower GPU memory utilization (40%)
C7	CPU Offload	Tests 4 GB CPU offload
C8	Determinism	Repeats baseline three times

3 Results

All nine configurations completed for both models. For each model, C0–C7 produce 20 outputs, while C8 produces 60 outputs because it repeats the baseline three times. Table 2 reports each non-baseline configuration against C0.

Table 2: Comparison against each model’s C0 baseline, averaged over 20 prompts.

Model	Config	Exact	BLEU	Edit Dist.	First Div.
Qwen2	C1/C2/C4/C6/C7/C8	1.000	100.0	0.0	–
Qwen2	C3 CUDA Graphs	0.250	69.7	154.2	28.0
Qwen2	C5 Batch Proc.	0.500	78.7	111.5	16.7
Qwen3	C1/C2/C4/C6/C7/C8	1.000	100.0	0.0	–
Qwen3	C3 CUDA Graphs	0.450	73.5	124.2	16.0
Qwen3	C5 Batch Proc.	0.300	73.3	142.2	22.1

The main result is consistent across models: memory-oriented changes and chunked-prefill variants are output-preserving, while CUDA graphs and batch processing are not. Qwen2 is more affected by CUDA graphs than by batching (25% vs. 50% exact match), while Qwen3 shows the reverse pattern (45% vs. 30% exact match). In both models, C8 exactly matches C0, confirming that the baseline itself is stable across repeated runs.

4 Discussion

The most useful distinction is between *memory-management* changes and *execution-path* changes. Prefix caching, reduced GPU memory utilization, CPU offload, and chunked prefill do not change outputs for either model in this setup. These changes affect storage, scheduling capacity, or prefill chunk size, but they do not appear to perturb the model’s token choices.

CUDA graphs and batch processing are different because they change how GPU work is scheduled or replayed. A plausible explanation is numerical sensitivity: different execution paths can introduce small floating-point differences, and greedy decoding can amplify a small logit-order change into a different token. This explanation is consistent with the observed first-divergence positions, but it is not directly proven here because we do not log intermediate logits or kernel-level numeric differences.

The two-model comparison makes the result more robust but also more nuanced. It is not simply

that one optimization is always “more unstable.” CUDA graphs diverge more on Qwen2, while batching diverges more on Qwen3. The practical lesson is therefore narrower and more actionable: exact reproducibility depends on both the model and the inference execution path, so both should be recorded when comparing LLM outputs.

This remains a focused single-system experiment: two small Qwen models, one GPU, one vLLM version, greedy decoding, and 20 prompts. Larger models, multi-GPU serving, sampling-based decoding, and direct logit comparison would be useful follow-up experiments.

5 Conclusion

Across both tested models, most vLLM optimizations preserve exact outputs, but CUDA graphs and batch processing can change generated text even with a fixed seed and temperature = 0. The central insight is not that LLM inference is generally unstable, but that exact output reproducibility depends on execution path as well as model and prompt.